

rung-ui: configure, queue and watch several continuous hosts

The acceptance scenario comes first. It is the run the owner makes on his phone to accept the whole thing; every later page exists to make it pass.

1. **Open.** On his phone, over the tailnet, he opens one https address and sees his instances, one row each: a state word, one line on what it is doing, requests used. One row is marked because it needs him.
2. **Open one.** He taps an instance. What it is doing now is the heaviest thing on the page, and it moves: turn text and tool calls appear as they happen.
3. **Ask why.** In Turns he finds a decision the host made, and who made it.
4. **Queue.** He adds a task. It shows under Waiting at once, is taken at the next boundary, and its answer appears in Turns.
5. **Configure.** He moves one model above another in the ladder and applies. The page says when it takes effect.
6. **See the effect.** A model-switch line appears, the next turn names the new model, the requests count carries on. No reload.
7. **Leave and return.** He locks the phone and comes back ten minutes later. The page catches up from where it stopped: nothing missing, nothing twice.

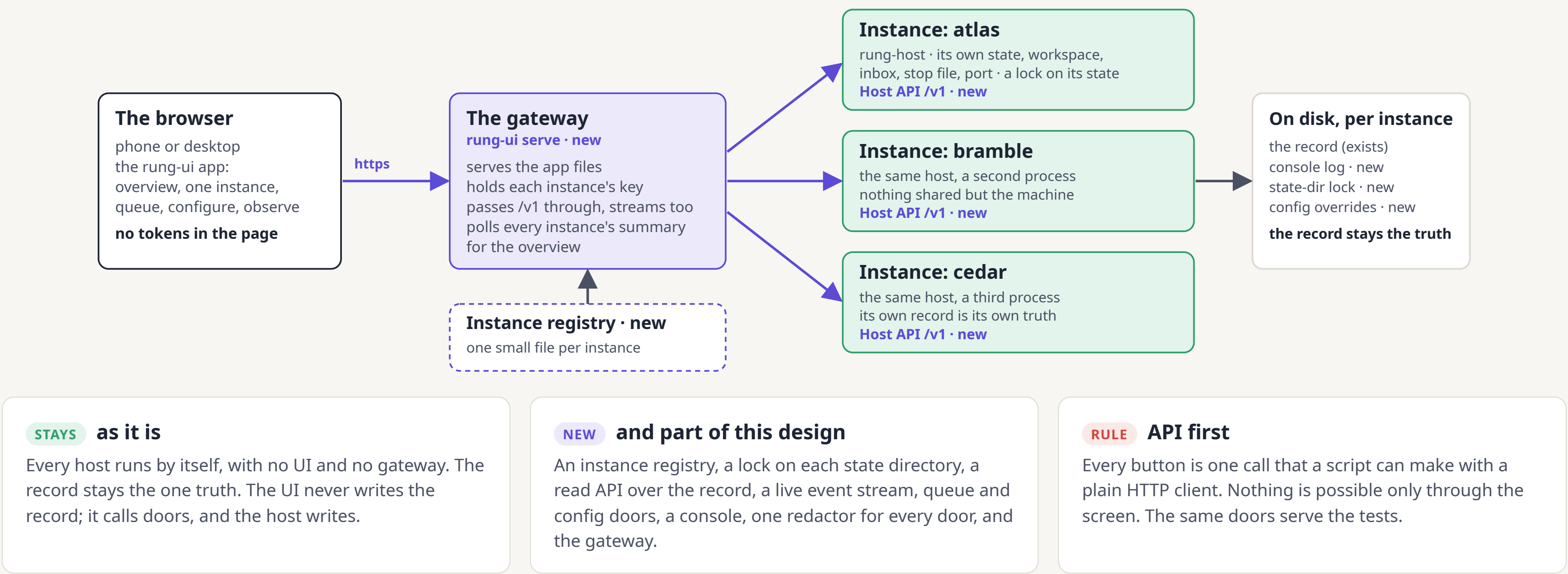
Each step has a check that can fail, and a slice that makes it pass

The checks are written before anything is built. Each is run first against the host as it is, where it must fail, and then after the change, where it must pass.

#	WHAT MUST BE TRUE	HOW IT IS PROVED	SLICE
1	The overview lists every registered instance. A stopped or dead one is listed, never missing. The state word comes from the record and the lock, not a guess.	Three registered instances; one is killed with kill -9 . The row reads Down within 30 seconds. Before the lock and registry exist this check cannot even start, so it fails first. The first screen holds at most 60 words of chrome at 390 and at 1920.	0, 4
2	A line the host records is on the screen within a second, in order, with the text as the model wrote it.	A real clock host writes a turn; the record time against the browser's receipt time, median and worst, through the gateway. The first live view is also run against one real free-model hour.	1
3	Every decision shows its question, its answer, who made it (rule or model, with cost) and why a rule stepped in.	Shown for each of the five decision families from a recorded run; a decision with no recorded reason fails the page test.	0
4	The add call answers only after the stimulus is on disk. The same call sent twice adds once. A kill right after the answer loses nothing.	An idempotency key repeated; kill -9 straight after the answer, then restart: the item has exactly one disposition (the host's existing restart gate, extended to the new door).	2
5, 6	Before applying, the page shows the difference and whether it takes effect now, at the next rollover, or only after a restart. Applying is recorded with who and when.	Move a ladder rung through the door; a config.changed line appears; the next turn's model matches the new order or a model.switch line explains why not. A restart-only key is refused with the reason and the old value untouched.	3
7	Reconnecting with the last seen number replays what was missed once, in order, even across a host restart.	Cut the connection for 10 minutes while turns run, and restart the host once; compare what the page holds with the record, line by line.	1
all	No real credential appears in any answer, stream line or console file.	A canary key is put in the host's environment and a tool is made to print it. Zero occurrences in every door, the stream and the console files. The check is written first and fails on the host as it stands.	0 on

One small gateway in front of any number of ordinary hosts

An instance is a rung host: one named agent that thinks continuously, with its own state, workspace, inbox, stop file and port. The UI is for those and nothing else.



Each screen has one thing to look at and one thing to do

The test: write the two sentences first, then ask of every element whether it serves one of them, would change what he does, and is not already obvious from where he is.

SCREEN	LOOK AT	DO	ONE REVEAL AWAY	NEVER ON THE SCREEN
Instances	One row each: state word, one line of what it is doing, requests used. The row that needs him is marked.	Open the marked row, or nothing: read	Model, spend, epoch, the instance's history	Charts. A count that changes nothing. "All is well" banners
One instance, Now	What it is doing now, live. Committed project and its done-when, or free time.	Add to queue (the one filled button)	Next (schedule, queue), Decided with a Why link, Health: context, cache, spend; recent turns, paged	Ids, hashes, token counts, raw JSON
Turns	The newest turn as it happens, text as written, tool calls as they run	Nothing: read. Search and filter are quiet	A turn's calls, cost, cache, decisions and who made them; the exact messages; the record line	A cap on how far back it goes. Nothing is cut: older turns are paged
Queue	What is being answered now; the next waiting item	Add (filled). Move and cancel are quiet row actions	Why an item waits (the decision, in words)	Kinds of item the host does not have
Configure	The model ladder, the rung in use marked	Apply N changes (always present; filled when something changed)	When each change takes effect; schedule, free time, budget, sandbox	Keys that need a restart, presented as editable
Console	The tail of host, agent and tool output, following	Search	Filter by source; page earlier; one long line in whole	A real credential, ever

Colour is attention only. **Amber**: he must act. **Red**: something failed. **One accent**: the one filled control and the focus ring. Everything else is ink and grey. State is a word first, never a colour alone. At most six tabs, with Now first.

INSTANCES

The overview: three rows and one of them is marked

Look at: the marked row. Do: open it, or nothing. No chart, no banner, no filled button.

Instances

Instances

Three running. One needs you.

atlas	Working	Tidying the notes index	357 of 1,000 requests	3 s ago
bramble	Free time	Nothing pulls it. Last real work 3 h ago.	198 of 1,000 requests	41 s ago
cedar	● Stuck	The model key was refused. Retrying every 15 min.	0 of 1,000 requests	9 min ago

Showing all 3 instances.

1920 wide. Full width, no centred column. The marked row uses amber, a dot and the word Stuck: the only colour on the page.

Looking changed one thing. The first draft had an “All systems reachable” line in the top bar. It said nothing he could act on, so it was removed. On the phone each row becomes three short lines, with the whole row as the tap target. The list says it shows all three, once, quietly.

Instances

Instances

Three running. One needs you.

atlas

Tidying the notes index

357 of 1,000 requests

Working

3 s ago

bramble

Nothing pulls it. Last real work 3 h ago.

198 of 1,000 requests

Free time

41 s ago

cedar

The model key was refused. Retrying every 15 min.

0 of 1,000 requests

● Stuck

9 min ago

Showing all 3 instances.

390 wide. The word and the dot say it; colour is not alone.

Now: what it is doing, live, and one button

Look at: the Doing now card. Do: Add to queue. Next, Decided and Health sit beside it and say little.

Instances / atlas

atlas

● Working. Turn 214 is running.

Add to queue

Now

Turns

Queue

Console

Configure

Doing now

Tidy the notes index

Committed 12 turns ago. Done when every note has an index entry.

● Live: writing a note, 38 s in.

Recent turns

213 · Wrote 3 index entries1 min ago

212 · Read the notes folder2 min ago

211 · Committed to the notes index3 min ago

Showing 3 of 214. Show more

Next

Stand-up at 14:30

2 queued

Decided

Held the digest for the next break.

Why

Health

Context 61% full. Cache 93%.

\$0.00 spent. 357 of 1,000 requests.

1920 wide. The card has the heaviest border; the one filled control is the button. Why opens the decision.

Looking changed three things. A Spend section was folded into Health (the first draft counted 73 words of chrome, over the budget of 60). “Turn 214” was said twice, in the line and in the card; it is now said once. “By rule, 12:04” became a Why link. Context and cache are one line; spend is the quiet line under it.

Instances / atlas

atlas

● Working. Turn 214 is running.

Add to queue

Now

Turns

Queue

Console

Configure

Doing now

Tidy the notes index

Committed 12 turns ago. Done when every note has an index entry.

● Live: writing a note, 38 s in.

Recent turns

213 · Wrote 3 index entries1 min ago

212 · Read the notes folder2 min ago

211 · Committed to the notes index3 min ago

Showing 3 of 214. Show more

Next

Stand-up at 14:30

2 queued

390 wide. The button takes the full width and the tabs fit without clipping.

Turns: watch it think, then page back as far as he wants

Look at: the newest turn, in progress at the top. Do: nothing, read. Search and filter are quiet.

Instances / atlas

atlas

● Working. Turn 214 is running.

Now

Turns

Queue

Console

Configure

Search turns

● Following

Turn 214 now

The index has 41 entries and the folder has 44 notes. Three are missing: the two from this week and the one about backoff. I will list them, then write the entries in order.

Tool

ws_list notes/ → 44 files

Tool

note_write index.md ... running

Turn 213 1 min ago

Wrote three index entries and checked the links. Two notes still lack a summary line.

Decision

Held the digest for the next break. By rule.

Turn 212 2 min ago

Read the notes folder and counted what the index covers.

Showing 3 of 214. Show 50 more · Show all 214

1920 wide.

A turn shows the text as the model wrote it, then each tool call; a decision shows who made it. Following means new turns arrive at the top.

Looking changed one thing. The first draft said “Following. New turns appear at the top.” and put calls and cache hit on every turn (67 words). The state is now one word, and calls and cache sit in the fold. Paging is a view, never a cap: *Show 50 more, Show all 214.*

Instances / atlas

atlas

● Working. Turn 214 is running.

Now

Turns

Queue

Console

Configure

Search turns

● Following

Turn 214 now

The index has 41 entries and the folder has 44 notes. Three are missing: the two from this week and the one about backoff. I will list them, then write the entries in order.

Tool

ws_list notes/ → 44 files

Tool

note_write index.md ... running

Turn 213 1 min ago

Wrote three index entries and checked the links. Two notes still lack a summary line.

Decision

Held the digest for the next break. By rule.

Turn 212 2 min ago

Read the notes folder and counted what the

390 wide.

Same anatomy. Long text wraps in whole; nothing is cut.

Mockup with invented data; census 39 words at 1920, 28 at 390

7 / 20

Queue: add, reorder, cancel, with what it is doing kept apart

Look at: what is being answered now and the next waiting item. Do: Add. A message is answered; a task may be committed to; a pull sits on its shelf for free time.

Instances / atlas

atlas

● Working. Turn 214 is running.

NowTurnsQueueConsoleConfigure

Add to the queue

Message ▾

What should it know or do?

Add

Being answered now

Which notes still lack a summary line?

Message · 2 min ago

Waiting

Draft the week's open questions

Task · 9 min ago

↑ ↓ Cancel

Stand-up: say what you are on

Message · 14:30

↑ ↓ Cancel

On its shelf

Look into why free models answer with one dot

Pull · 1 d ago

Remove

1920 wide.

Three kinds in one list structure: Message, Task, Pull. The columns on the right line up across rows.

Looking changed one thing. At 1920 the right-hand columns did not line up from row to row, so the eye could not scan the ages. They are now fixed-width. Cancel on the item being answered is not offered as a quiet action: it needs a decision card, because it cuts the running turn.

Instances / atlas

atlas

● Working. Turn 214 is running.

NowTurnsQueueConsoleConfigure

Add to the queue

Message ▾

What should it know or do?

Add

Being answered now

Which notes still lack a summary line?

Message · 2 min ago

Waiting

Draft the week's open questions

Task · 9 min ago

↑ ↓ Cancel

Stand-up: say what you are on

Message · 14:30

↑ ↓ Cancel

On its shelf

Look into why free models answer with one dot

Pull · 1 d ago

Remove

390 wide.

Move up, move down and Cancel are 44-point targets.

Configure: the ladder first, and one button that says what it does

Look at: the model ladder with the rung in use marked. Do: Apply 1 change. The line above the button says when it takes effect.

Instances / atlas

atlas

● Working. Turn 214 is running.

Now

Turns

Queue

Console

Configure

Model ladder

1	stealth/space-bunny-alpha	Expired
2	qwen/qwen3.8-27b:free	In use
3	nvidia/nemotron-3-ultra-550b-a55b:free	
4	google/gemma-4-31b-it:free	

Free time

☐ Answer idle turns in one line

Budget

Requests a day1,000A minute20

Schedule

Stand-up at 14:30, firm

Sandbox

/home/zung/atlas/workspace

Takes effect on its next turn.

Apply 1 change

1920 wide.

The ladder is one decision, so its options sit together and in order. Schedule, free time, budget and sandbox follow.

Looking changed two things. The first draft explained the ladder in a sentence and listed the tool groups; at 1920 that made 86 words of chrome. Both were cut from the page; the ladder explanation is to return as a one-click reveal. The apply sentence went from two sentences to one. The sandbox path is shown, not editable: it needs a restart.

Instances / atlas

atlas

● Working. Turn 214 is running.

Now

Turns

Queue

Console

Configure

Model ladder

1	stealth/space-bunny-alpha	Expired
2	qwen/qwen3.8-27b:free	In use
3	nvidia/nemotron-3-ultra-550b-a55b:free	
4	google/gemma-4-31b-it:free	

Free time

☐ Answer idle turns in one line

Budget

Requests a day1,000A minute20

Schedule

390 wide.

Long model names wrap in whole.

Mockup with invented data; census 54 words at 1920, 44 at 390

9 / 20

Console: everything the host and its tools print, with secrets removed in code

Look at: the tail, following. Do: search. Lines are whole; long ones fold in place; nothing is cut.

Instances / atlas

atlas

● Working. Turn 214 is running.

NowTurnsQueueConsoleConfigure

Search the console

All output

● Following. Secrets show as [redacted].

14:02:11hostturn 214 started: free → committed, rung 2 of 5, epoch 31

14:02:11agentPOST https://router.example/api/v1/chat/completions 200 in 3.1 s, 22,114 prompt tokens, 21,403 cached

14:02:12toolws_list notes/ → 44 files

14:02:14toolenv: RUNG_HOST_OPENROUTER_API_KEY=[redacted]

14:02:15agentstderr: warning: result of note_write kept verbatim (12,884 bytes)

14:02:49hostturn 213 ended: answered, 3 calls, cost \$0.00, hash 9f3a...c21

Showing the last 6 of 18,420 lines. Show 500 earlier · Show all

1920 wide.

Time, source, text. A secret reads [redacted] before it is stored and again before it is sent.

Looking changed one thing. In the first phone capture the last tab, Configure, was cut off to “Con”. The census did not catch it; looking did. The tabs now take less padding at 390, and the census gained a check that the active tab is fully visible. Console lines are paged: Show 500 earlier, or Show all.

Instances / atlas

atlas

● Working. Turn 214 is running.

NowTurnsQueueConsoleConfigure

Search the console

All output

● Following. Secrets show as [redacted].

14:02:11hostturn 214 started: free → committed, rung 2 of 5, epoch 31

14:02:11agentPOST https://router.example/api/v1/chat/completions 200 in 3.1 s, 22,114 prompt tokens, 21,403 cached

14:02:12toolws_list notes/ → 44 files

14:02:14toolenv: RUNG_HOST_OPENROUTER_API_KEY=[redacted]

14:02:15agentstderr: warning: result of note_write kept verbatim (12,884 bytes)

14:02:49hostturn 213 ended: answered, 3 calls, cost \$0.00, hash 9f3a...c21

Showing the last 6 of 18,420 lines. Show 500 earlier ·

390 wide.

A line wraps; a very long one folds with Show whole line.

Mockup with invented data; census 32 words at 1920, 30 at 390

10 / 20

Criteria first, a census written before the pages, then looking at every capture

The census counts what a script can. It is a flag, not a verdict, so every capture was also looked at, at both widths.

The rules, written before the mockups

- **One look, one do** for every screen, written as two sentences.
- **Colour is attention:** amber for a human decision, red for failure, one accent for the one filled control.
- **One decision card** at a time, when something truly needs him.
- **Nothing appears or disappears after load.** Empty and loading regions keep their place and their heading.
- **Folds, not truncation. Paging, not caps.** Every list says how many and offers more or all.
- **Plain words.** Say *queue, turn, ladder*; no ids or hashes in the main view.
- **Each fact once.** Four text sizes at most. No key hints drawn on buttons.
- **Looked at** at 390 and at 1920 before it counts as done, and **proved live** against a running host.

Fail first: the budgets below were written before the pages. The first draft failed them on 3 pages at 1920 and 1 at 390; the fixes were made, and the census then passed on all 12 captures.

The census, first draft and now

PAGE	WORDS AT 1920	WORDS AT 390
Instances	42 → 39	42 → 39
Now	73 → 60	55 → 46
Turns	67 → 39	56 → 28
Queue	55 → 51	49 → 45
Configure	86 → 54	62 → 44
Console	36 → 32	34 → 30

Budgets, per page, first screen. Words of chrome (not counting what he came to read) at most 60. One filled control. At most 4 text sizes. No sentence said twice. No key hints. No text under 14 pixels. One level-one heading. At 390: 44-point tap targets, no sideways scroll, active tab fully visible.

Found by eye, not by census: a status line that said nothing; a fact said twice in different words; a clipped tab. The first two cannot be counted; the third now can.

What the UI needs, what the host has, and the piece that closes each gap

Read from the host's design note and its code. The host runs one agent per process; the record is its only log.

THE UI NEEDS	THE HOST TODAY (read in the code and design note)	THE PIECE THAT CLOSES IT
List the instances	Nothing lists them. Each instance exists only as a separate config: its own state, workspace, inbox, stop file and ACP port.	H2 registry: one small file per instance, written at start, kept after stop
Two never share a state	No lock. A second host started on the same state directory would write into the same record.	H1 a lock held for life on the state directory; a clean refusal naming the holder
Is it alive, and doing what	An ACP status call (turn, mode, project, epoch, model, ladder, quota, degraded, pending) and a plain-text report, over ACP only; the process pings a watchdog.	H3 a summary door over HTTP with one state word
Read the past	The NDJSON record on disk, one line per event, gap-free numbers. No read API. The turn text and messages are in it verbatim.	H3 a paged read door and turn, decision, pack and spend folds
Watch live	An internal hook feeds the ACP bridge. An observer channel sees only the output of work it owns. Nothing streams record lines. A model call is recorded when it ends.	H5 an event stream; H5b short-lived deltas, if he wants them (question 4)
Queue	A directory of message files, an ACP stimulus call (on disk before its answer), cancel that withdraws a waiting prompt, and an owner calendar call. Status counts waiting items but does not list them. No order to change.	H6 list, add, cancel and move doors
Configure	A YAML file read once at start; unknown fields refused; the ladder is configuration, re-listed every six hours. No way to change a running host.	H7 validate and apply doors with effect classes, recorded
A console	No such thing. The host's own output goes to the system journal; the record holds no process output.	H8 a console sink and doors
No credentials out	A pattern redactor exists and is used for short gists and memory. The record is verbatim on purpose, so the cached prompt prefix can be rebuilt byte for byte.	H4 one redactor at every door and in the console sink
Metrics, MCP	None. They are not needed: the record is the log, and the UI reads it.	No piece. Not added

Every button is one call; these are the doors

All on the host's own HTTP listener, next to ACP, behind the same bearer tokens. Reads need a read-only token or the owner's; writes need the owner's.

GROUP	CALL	WHAT IT DOES	TODAY
Who	GET /v1/summary	The overview row: id, name, state word, one line of what it is doing, requests used and spend this UTC day, what needs the owner	new
	GET /v1/status · /v1/report	The ACP status as JSON; the plain-text report	exists over ACP
Read	GET /v1/record	Lines, paged: offset, limit, total, next. Filter by kind, turn, text. Newest first or oldest first	new
	GET /v1/turns · /v1/turns/{n}	One summary per turn; one turn in full with its messages, calls, decisions	new
	GET /v1/decisions	The decision desk's records by family, with who decided and why	new
	GET /v1/pack	Epoch, context against its budget, rollovers and their cause, cache hit and efficiency, cache breaks	new
	GET /v1/spend	Requests, tokens and cost by UTC day and by model	new
Live	GET /v1/events?after=N	Server-sent events: each record line as it is written, numbered; resumes from N or Last-Event-ID; short-lived deltas	new
Queue	GET · POST /v1/queue	List (being answered, waiting, shelf, schedule); add a message, task, pull or calendar entry, with an idempotency key	parts exist
	DELETE /v1/queue/{id} · POST .../move	Cancel; move before another item	cancel exists; move new
Config	GET /v1/config	The effective config, secrets as the name of the variable that holds them, each key with its effect class	new
	POST .../validate · PUT /v1/config	The difference and class for a change; apply it, recorded as who, when, old, new	new
Console	GET /v1/console · .../stream	Paged and live process output; search; source filter	new
Control	POST /v1/stop · /v1/release	The owner's stop and release	exist over ACP

The gateway adds two more: **GET /api/instances** (the registry and each summary) and **/api/i/{id}/v1/...** (a pass-through that adds that instance's key and streams events through). Lists follow one paging rule: **offset, limit, total, next**; limit has no maximum; total is exact.

The record is the event log; the stream is the record, followed

A stable, resumable stream needs one durable source. The host already has it: an append-only, gap-free, fsynced record.

Server-sent events, not a socket

- One direction is all the screen needs; every command is an ordinary call.
- Each event carries its record number as its id. A reconnect sends the last one it saw, and the host replays from the record. A host restart changes nothing.
- Passes a tailnet https proxy untouched. A socket can be added later if something needs two-way traffic.
- A slow reader is dropped and reconnects; nothing is lost because the record holds it all.
- A comment line every 15 seconds keeps the path open.

What is stable

- The **kinds and fields** of record lines are the contract. They are listed in one table in code, the design note is generated from it, and a test fails if they drift.
- Changes inside version 1 only add. A removed or renamed field needs version 2.
- Short-lived **delta** events carry the text and tool progress of the turn that is running. They have no number, are never replayed and never enter the record, so the record stays reproducible (question 4).
- The host's start line states the contract version.

Credentials stay in

- **One redactor, three places:** the record door, the stream, and the console sink. It removes the exact value of every key the config names by variable, plus the known key shapes already recognised for memory.
- It replaces; it never shortens or drops. No length cap anywhere.
- The record on disk stays verbatim, because the cached prompt must be rebuilt byte for byte. The door redacts on the way out.
- Written first as a failing test: a canary in the environment, a tool that prints it, zero hits across every door and file.

The console is a second, smaller log

The record has no process output, so a console sink writes its own numbered lines: **host** (the host's own log lines), **agent** (the model route's request and answer lines) and **tool** (what a tool process printed, as it prints). It is redacted before it is written, because nothing needs it verbatim. Same paging, search and stream as the record. A very long line is stored whole and folded in the screen.

A registry file and a lock per instance; the state word is derived, never typed

The overview has to be true after a crash, so it never trusts what an instance says about itself: it joins the record with whether the lock is held.

The registry

- One small file per instance in a registry folder under the rung home. Fields: id, name, state directory, config path, workspace, address, process number, started, version.
- Written when the host starts and kept when it stops, so a stopped instance is still listed.
- The host binds its address on any free port and writes the real one, so ports never collide by hand.
- A command lists them (**rung-host ls**); the gateway reads the folder.

The lock

- A lock file in the state directory, held by the process for as long as it lives.
- A second start on the same state refuses, exits with the code the host already uses for a bad start, and names the holder. The check is written first: today the second start succeeds.
- Alive means the lock is held. A dead host drops it by dying, so a crash cannot leave a false Working.

Start and stop

Stop and release go through the doors the host already has. Starting is the system's job: one templated user unit, one instance per name. A stopped row shows the command to start it. A supervisor in rung is a question for the owner (question 2).

The seven state words

WORD	WHEN	COLOUR
Working	The lock is held and the agent is committed to a project	none
Answering	The lock is held and the turn answers a waiting item	none
Free time	The lock is held and nothing pulls it. If its last three free turns made no tool call, the line says it is idle	none
Waiting	Slowed by the world: pacing, quota or a provider backoff. The line says until when	none
Stuck	A refused key, a copy loop the host had to break, a commitment with no progress for a set number of turns, a turn past its bound, or a missed watchdog	amber
Down	The lock is free and the last record line is not a halt: it died	amber
Stopped	The lock is free and the last line is a halt	none

Needs him is one more mark on a row: a message from the agent to its owner not yet read, or Stuck, or Down, or a spend cap reached. Nothing else raises an alert. Free-model hosts show requests used of the day's quota beside spend, because on the free ladder the quota, not money, is the meter. The thresholds are settings.

A directory in the rung repo, built like a normal web app, proved with real captures

Small and boring on purpose. The same pull request can change a door and the screen that uses it.

Where it lives

- A **rung-ui** directory in the rung repo: the app, its tokens and its tests. A small **gateway** crate beside the host crates, not published.
- Both are product, not kernel: nothing in the ladder crates changes, and no consumer is named anywhere (the repo's guard already checks this).
- Why not its own repo: the door contract, its gates and the screens move together, and the docs and guard checks already run here. The cost is a second toolchain in CI (question 1).

What it is built with

- React, TypeScript and Vite; a query cache for the reads, and a small hook that merges the live stream into it by record number.
- **Design tokens as plain CSS variables**, rung's own copy. The stylesheet behind these mockups is the seed: four text sizes, ink and grey, one accent, amber and red held back.
- No component kit at first. Dialog and select primitives only if a screen needs them.
- Reading measure for prose; lists and tables use the full width.

Serving

- The gateway listens on loopback and embeds the built app. Tailscale's own serve command puts it on tailnet https, so the phone opens one address.
- One systemd user unit. It reads the registry folder and a tiny file naming which variable holds each instance's key. Keys never reach the browser.
- No login in the prototype; the tailnet is the boundary (question 6).

Testing

- Unit tests for the folds, run on the recorded runs already in the repo.
- **Playwright captures at 390 and 1920 of every screen**, against two or three real hosts on a real clock. A capture is looked at before a slice is done.
- The **census** from this note becomes a CI gate. A new screen lands with its budget and a recorded first failure.
- Every slice ends with a live proof (the next pages), never with a green mock.

SLICES

Five slices, each ending in something he can use

Effort is a coding agent's days, rough, one host builder and one UI builder working side by side. Each slice is judged by a live proof, then by looking at its captures.

SLICE	THE LOOP IT CLOSES	HOST PIECES	UI	LIVE PROOF	EFFORT
0 • Observer on a recorded log	Read-only. He opens the instances recorded on disk, reads turns, decisions, context and spend, and cannot change anything.	H3 read doors and folds, H4 redactor, H9 gateway	Scaffold, tokens, census; Instances, Now, Turns	The repo's recorded live runs open in the page; the canary check passes; captures at 390 and 1920	Host 10, UI 8
1 • Live instance view	He opens a running instance and watches it think.	H1 lock, H2 registry, H5 events, H8 console	Live Now, Turns following, Console, state words	Two real-clock hosts; record time to screen time; reconnect and replay; one real free-model hour	Host 11, UI 5
2 • Queue	He adds, moves and cancels from the phone.	H6 queue doors	Queue tab, Add to queue	Add, answered in Turns; the same call twice adds once; kill after the answer loses nothing	Host 5, UI 3
3 • Configure	He changes the model ladder and sees the effect.	H7 config doors and overrides	Configure tab	Move a rung, see config.changed, then the next turn on the new model, live	Host 7, UI 4
4 • Side by side	Three instances at a glance, tailnet https, the phone captures.	Thresholds, needs-him marks, unit template	Overview polish, alerts	Three hosts, one killed: Down inside 30 seconds; the phone scenario run once end to end	Host 2, UI 3, ops 1

Who builds

A host builder (Rust, in the host and gateway crates) and a UI builder (TypeScript, in rung-ui), each on their own branch and pull request through the checking pipeline, one slice at a time. The design owner reviews the captures before a slice is called done.

Order

Slice 0 first and alone on the UI side. H1, H2 and H3 do not depend on any question and can start in parallel with it. Slices 1 to 4 follow in order; 2 and 3 can overlap once slice 1 is done. About 59 agent-days in all.

Nine plain-engineering pieces, most of them independent

Each is a small pull request on its own, with its check written first. None needs the screen.

#	PIECE	DONE WHEN (THE CHECK THAT FAILS FIRST)	DAYS	NEEDS	CAN START NOW
H1	State-directory lock	A second host on the same state exits with the bad-start code, names the holder, and leaves the record's last line unchanged	1	nothing	yes
H2	Registry and rung-host ls	Start writes the file, stop keeps it, a killed host reads Down; ls shows all three states	2	H1	yes
H3	Read doors and folds	Turn, decision, pack and spend folds of a recorded run equal what the live host computes at each turn end (the host already hashes its projection); paging totals exact	5	nothing	yes
H4	Redactor	The canary check: zero hits in every door, stream line and console file; no output shorter than its input except by the replaced value	2	nothing	yes
H9	Gateway	Serves the app, passes /v1 and streams through, adds the key, lists the registry; no key in any page or response	3	H3 shape	yes
H5	Event stream	Replay after a cut equals the record; a slow reader does not slow the host; the first line reaches a client within a second of being written. Needs the HTTP listener to carry routes beside ACP	4 (+3 for deltas)	H3; question 4	after the answer
H6	Queue doors	List matches the host's waiting items; add on disk before the answer; idempotent; cancel withdraws; move recorded and respected, the desk's limits still winning	5	H3; question 3	after the answer
H7	Config doors	Validate equals startup validation; a hot key changes the running host at the next boundary; a restart-only key is refused; base file untouched	7	H3; question 5	after the answer
H8	Console sink	Host, agent and tool output numbered and redacted before written; a 12 MB tool output stored whole and paged; search and follow work	4	H4	with slice 1

Six questions, most blocking first

Default if unanswered: only slice 0, the read-only observer, and the host pieces that no question touches (H1 to H4, H9) are built. Nothing live, queued or configurable is built until he answers.

1. Where does rung-ui live?

In the rung repo, or in its own repository?

Recommendation. In the rung repo, as a directory and a gateway crate. The door contract, its checks and the screens change in one pull request.

What the answer changes. A second toolchain in CI and a mixed-language repo, against two repos that must agree on a contract by copy.

2. Who starts and stops instances?

Should the page only stop and release, with starting left to the system's unit manager?

Recommendation. Yes for now: one templated unit, and the page shows the command to start a stopped one. No supervisor in rung yet.

What the answer changes. Whether the page ever has a Start or Create instance button. A supervisor adds process management and its failure modes.

3. What does moving a queue item mean?

May the owner's move override the decision desk?

Recommendation. No. A move orders items waiting for the same boundary; owner items still go first; the desk's limits still win, and the page says so.

What the answer changes. The admit code and a new check. If a move could override everything, one tap could starve a commitment.

4. How live is live?

Stream the model's partial text and tool progress as it is written (short-lived, never recorded), or show only finished calls?

Recommendation. Stream partial text. Watching it think is the most important thing he asked for. The record keeps only the finished text, so replays stay exact.

What the answer changes. The engine adapter must stream, which it does not today: about 3 more days. He then sees the model's text as it forms. Finished calls only is simpler, with a few seconds' lag.

5. How are config changes stored?

Edit the base config file, or layer an overrides file over it?

Recommendation. An overrides file in the state directory, applied over the base. The base file, which fleet tooling places, never drifts, and a value set in the page shows as set there, with a way back.

What the answer changes. Where the truth for a changed value lives, and whether the page and the config manager can fight.

6. Who may reach it?

Is the tailnet the only boundary, with no login in the app and keys held in the gateway?

Recommendation. Yes, plus a read-only token role so a view-only link is possible later. Prototype posture: protect real credentials and data, nothing more.

What the answer changes. Anyone on the tailnet can queue and configure. A login would add a user store and sessions.

What was read, what was measured, and what is still a guess

A design note, so most of it is proposal. These lines say which parts rest on evidence.

VERIFIED read in the code or notes • The host's loop, record kinds, ACP calls, status fields, ladder and listing, governor, pack and desk, as in its design note and source. • The host has no registry, no state lock, no read API over HTTP, no record stream, no queue list or order, no config door and no console. • The ACP HTTP listener already serves server-sent streams for ACP. • The record is numbered without gaps, written before use, and recovers from a torn last line.	MEASURED by running • Six real HTML mockups, captured with Playwright at 390 and 1920 (12 images), each looked at. • The census, run on all 12 captures: first draft, then final. Both outputs are committed. • Nothing else. No host was changed or run for this note.	UNKNOWN or inferred • Latency of a stream through the tailnet proxy to a phone. • Which config keys can change on a running host without a restart; the classes here are read from the code, not tried. • The cost of searching a very large record; an index may be needed. • Whether the engine adapter can stream partial text (question 4). • The state-word thresholds, and what the agent's own messages to its owner look like in practice. • Whether free-model quota, rather than cost, should be the headline meter.
--	---	--

Everything for this note lives in **docs/reports/rung-ui-design-v1/**: the build recipe, the deck source, the mockup pages and their capture and census scripts, the 12 captures and both census outputs. The mockups are static pages with invented data, not the app.